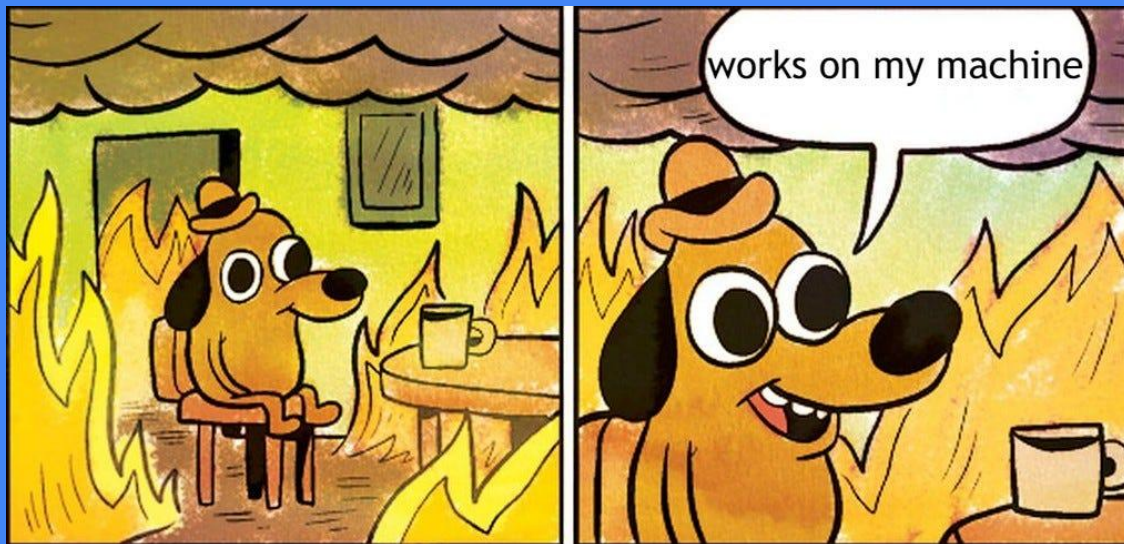


Contenedores - 101

De localhost a producción

02/10/2026



About.me



Eduardo Orta Salazar

Software Engineer | Linux Geek | Coffee Lover |
Co-Organizer at Calzada Code | SpaceXAI Ambassador



@Lalo_Salazar



esalazar.sizr@gmail.com



@eduardo-orta-salazar

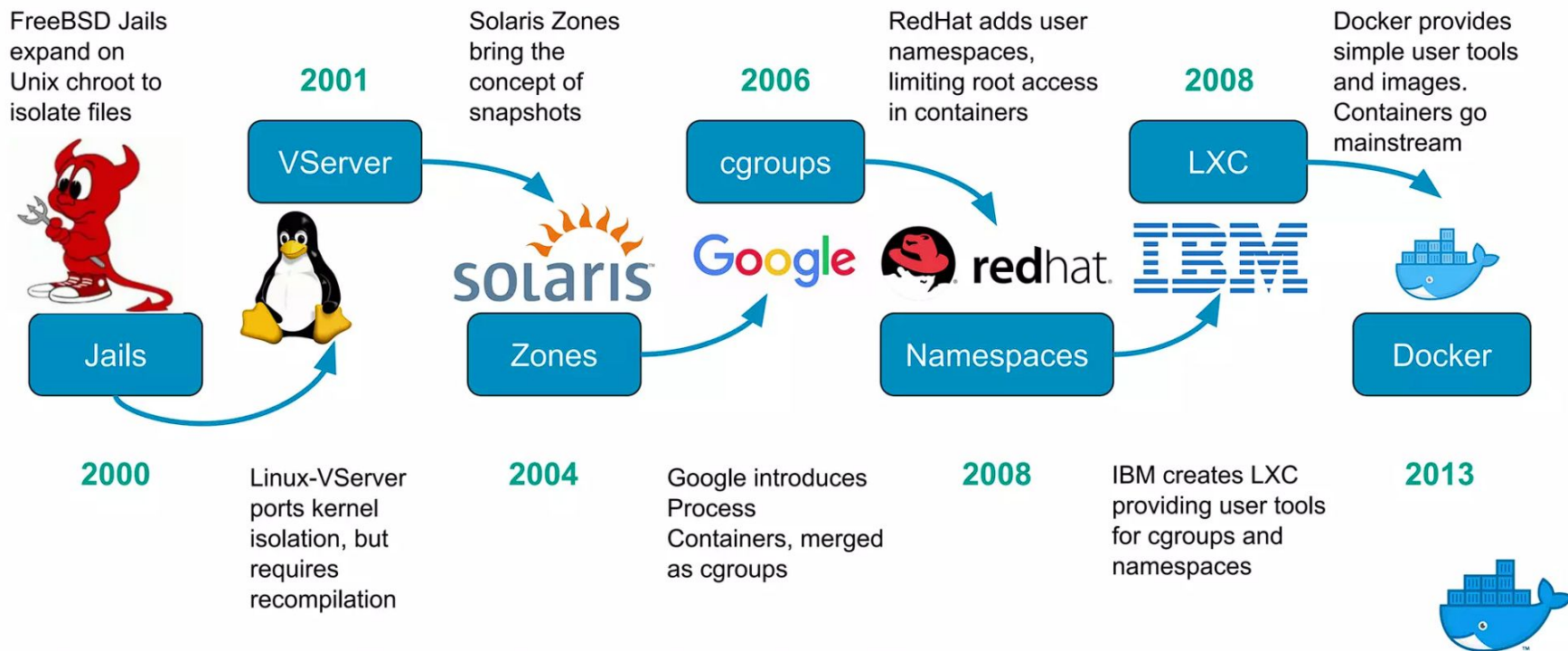


@lalo_salsa_katsun

Un poco de historia

- **1979 - UNIX chroot:** Permite cambiar la raíz visible del filesystem de un proceso. Es uno de los primeros antecedentes de aislamiento.
- **2000 - FreeBSD Jails:** Extiende el aislamiento a procesos, usuarios, filesystem y red. Se acerca mucho más al concepto moderno de contenedor.
- **2001 - Linux-VServer:** Lleva la virtualización a nivel de sistema operativo al ecosistema Linux, aislando recursos y entornos.
- **2004–2005 - Solaris Zones:** Sun desarrolla una forma madura de virtualización a nivel de SO, con aislamiento de procesos y recursos.
- **2005 - OpenVZ:** Populariza la virtualización basada en contenedores en Linux.
- **2006–2008 - cgroups:** Linux incorpora mecanismos para controlar y contabilizar recursos como CPU y memoria de grupos de procesos.
- **2008 - LXC (Linux Containers):** Combina mecanismos de aislamiento del kernel para crear entornos de contenedores prácticos en Linux.
- **2013 - Docker:** Hace que construir, empaquetar y ejecutar contenedores sea mucho más sencillo para desarrolladores. Docker se convierte en el gran catalizador de adopción.
- **2014 - Kubernetes:** Google presenta Kubernetes, llevando los contenedores hacia la orquestación a gran escala.
- **2015 - Open Container Initiative (OCI):** Se empiezan a estandarizar formatos de imágenes y runtimes para evitar depender de una implementación específica.
- **2016–hoy - containerd, runc, Kubernetes, ECS, etc.:** El ecosistema se separa en componentes especializados y los containers se convierten en una unidad fundamental de despliegue cloud-native.

Docker no inventó los contenedores pero si los popularizó :)



Algunos conceptos importantes

CHROOT: Es una operación de los sistemas operativos Unix/Linux que cambia el directorio raíz operativo para un proceso en ejecución y sus subprocesos.

JAILS: Aislamiento a nivel SO que virtualiza un entorno donde los procesos poseen sus propios usuarios, redes (direcciones IP) y variables de entorno, sin poder interactuar ni dañar el sistema operativo del host.

CGROUPS: Característica del kernel de Linux que administra, limita y asigna el uso de recursos físicos del sistema entre grupos de procesos. No se encarga del aislamiento visual de los datos, sino de regular el consumo de hardware.

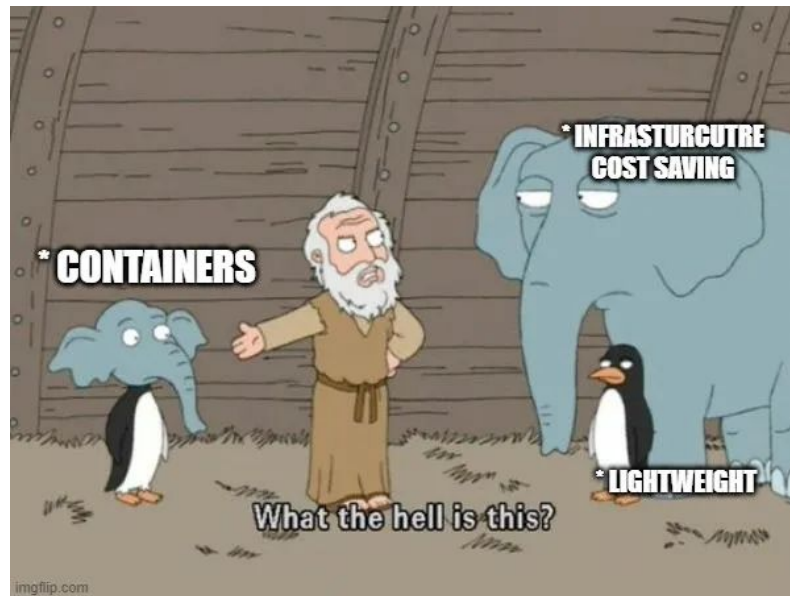
NAMESPACES: Función del kernel de Linux que aísla visualmente los recursos del sistema operativo para que un grupo de procesos crea que tiene una instancia exclusiva de ellos. Mientras que los cgroups restringen cuánto puede usar un proceso, los namespaces restringen qué puede ver ese proceso.

RUNC: Herramienta de línea de comandos ligera y de bajo nivel diseñada específicamente para generar y ejecutar contenedores en Linux de acuerdo con la especificación de la OCI (Open Container Initiative). Su única función es tomar un sistema de archivos raíz (creado a partir de una imagen de contenedor) junto con un archivo de configuración estándar (**config.json**), y usar funciones nativas de Linux como **namespaces** y **cgroups** para dar vida al contenedor.

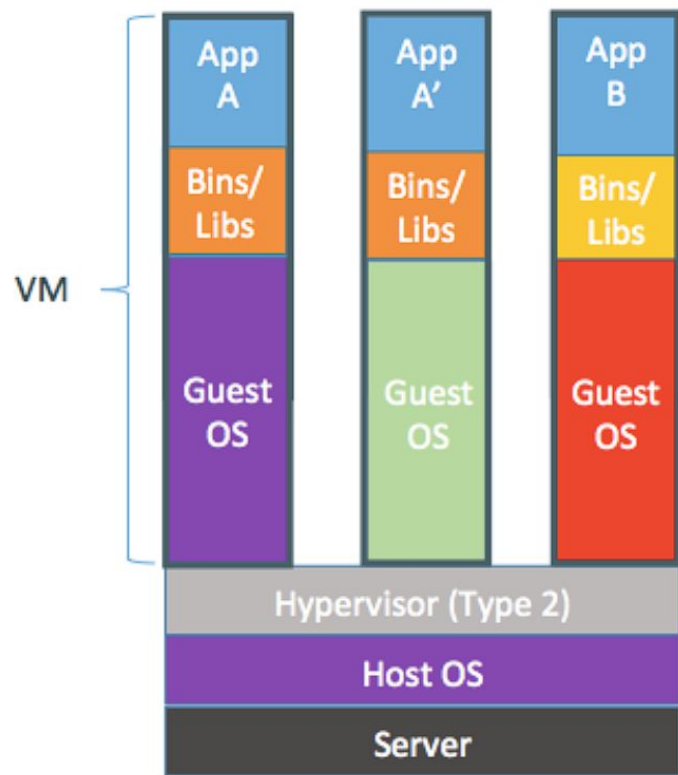
CONTAINERD: Runtime que se encarga de gestionar todo el ciclo de vida de los contenedores en un host. Sus funciones principales incluyen la transferencia y almacenamiento de imágenes de contenedores, la ejecución de los contenedores (comunicándose con herramientas de bajo nivel como **runc**), la gestión del almacenamiento local y la supervisión de las redes del contenedor.

Ahora si, ¿Que es un contenedor?

En pocas palabras, un contenedor es un proceso más en tu máquina que ha sido aislado de todos los demás procesos de la máquina anfitriona. Dicho aislamiento se basa en los **namespaces** y los **cgroups** del kernel, características presentes en Linux desde hace mucho tiempo.

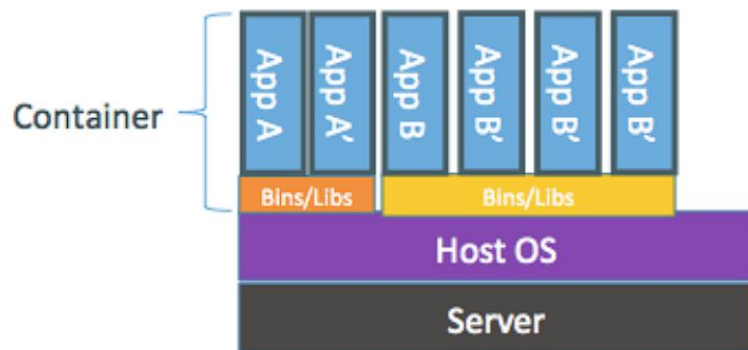


Virtual Machines VS Containers



Containers are isolated,
but share OS kernel and, where
appropriate, bins/libraries

...result is significantly faster deployment,
much less overhead, easier migration,
faster restart



En mi local funciona!

El problema:

Diferentes runtimes, dependencias, configuraciones, etc.

Necesidad:

Forma reproducible para ejecutar software.



DEMO :S



Docker, Docker



Yes papa



Eating RAM ?



No papa



Telling lies ?



No papa



Open your mouth!



Process Name	Memory ▾
com.docker.hyperkit	9.06 GB

Buenas practicas

✓ Imagen pequeña

Preferir imágenes como alpine o imágenes oficiales minimalistas reduce tamaño, tiempo de descarga y superficie de ataque. (siempre se puede usar FROM: scratch)...

✗ Imagen gigante

✓ .dockerignore

Evita copiar archivos innecesarios al contexto de build, como node_modules, .git, logs, secretos o archivos temporales.

✗ COPY de todo

✓ Secretos fuera

Passwords, API keys y tokens no deben estar en el Dockerfile ni hacer COPY sin revisar qué se está incluyendo. Usar variables de entorno o un gestor de secretos.

✗ Secretos en Dockerfile

✓ Usuario no-root

Si la aplicación es comprometida, un usuario sin privilegios limita el impacto dentro del contenedor.

✗ Ejecutar como root

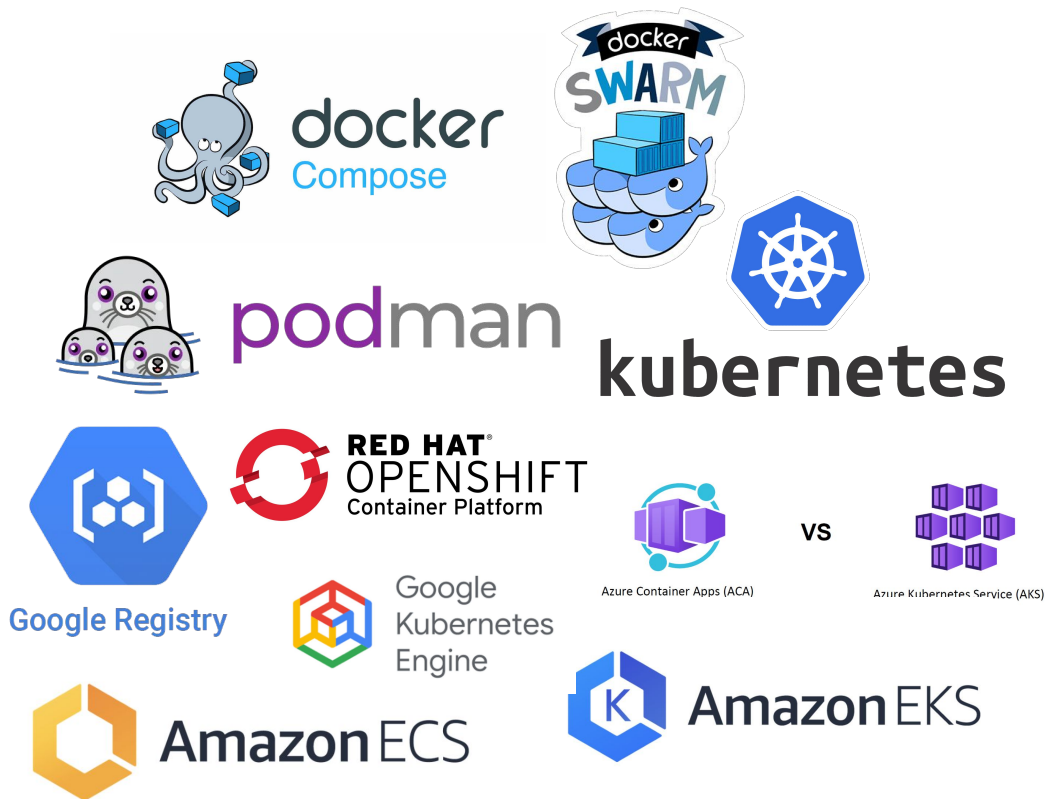
✓ Versiones fijas

Preferir node:22-alpine o, para mayor reproducibilidad, un digest, en lugar de depender de latest. Facilita reproducir exactamente el mismo entorno.

✗ Depender de latest

¿Qué sigue después?

<https://roadmap.sh>



Referencias

<https://container.training/intro-selfpaced.yml.html#1>

<https://www.docker.com/101-tutorial/>

<https://sergiobelkin.com/categories/contenedores/>

<https://medium.com/@dmosyan/linux-cgroups-explained-how-containers-use-it-c99eebb8c9c6>

<https://www.hackstack.org/understanding-linux-cgroups/>

<https://www.linuxfoundation.org/blog/blog/a-brief-look-at-the-roots-of-linux-containers>

<https://www.youtube.com/watch?v=UZpyvK6UGFo&list=PLqRCtm0kbeHAep1hc7yW-EZQoAJqSTgD->

Preguntas?



Por si acaso...

